

🪄 Genie — Enterprise Chat User Guide

คู่มือการใช้งาน Enterprise AI Assistant สำหรับทีม DIO

📋 สารบัญ

- 1 📖 วิธีใช้ Genie — Open WebUI Wiki
แชท, แนบไฟล์, ฟังก์ชันพื้นฐาน, Tips & Tricks, Use Cases
- 2 📚 Knowledge Base
สร้างและจัดการฐานความรู้สำหรับ Agent
- 3 💬 Prompts
เทมเพลตคำสั่งสำหรับเรียกใช้ซ้ำ
- 4 🤖 ขั้นตอนสร้าง Agent
สร้าง Model ที่มี Tool ติดตัว
- 5 🧠 Skills
ความสามารถเฉพาะทางของ Agent
- 6 🔧 ขั้นตอนเพิ่ม Tool (Advance)
เพิ่ม Enterprise Search Tool

วิธีใช้ Genie — Open WebUI Wiki

คู่มือการใช้งาน Genie AI Assistant สำหรับพนักงานหาดทิพย์

แชทกับ Genie

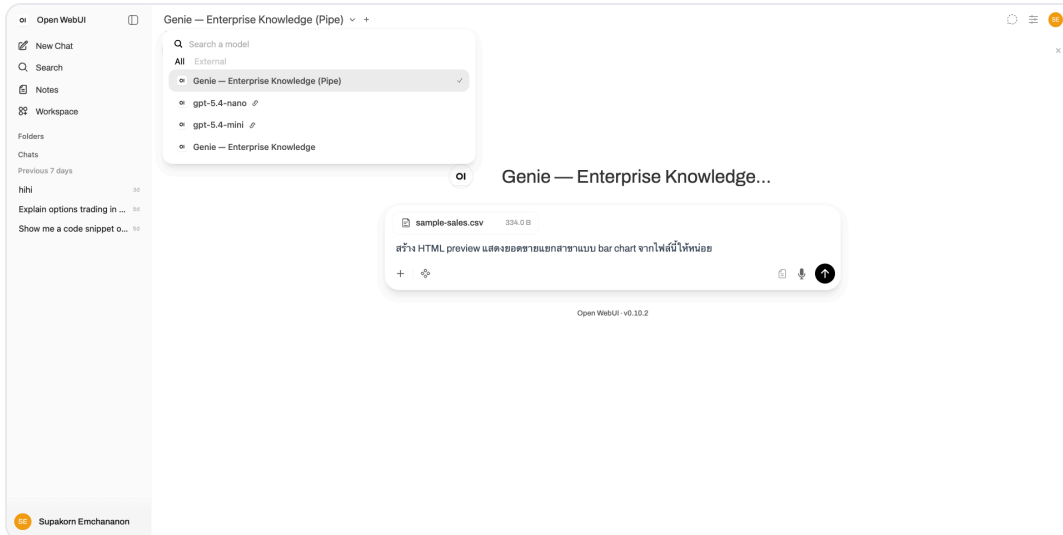
- เปิด **New Chat** — เลือกโมเดลจาก dropdown ด้านบน
- พิมพ์คำถามเป็นภาษาไทยหรืออังกฤษก็ได้
- Genie ค้นหาจากฐานข้อมูลเอกสารภายในก่อนตอบ
- คำตอบจะอ้างอิงชื่อเอกสารต้นทาง

แนบไฟล์ถาม

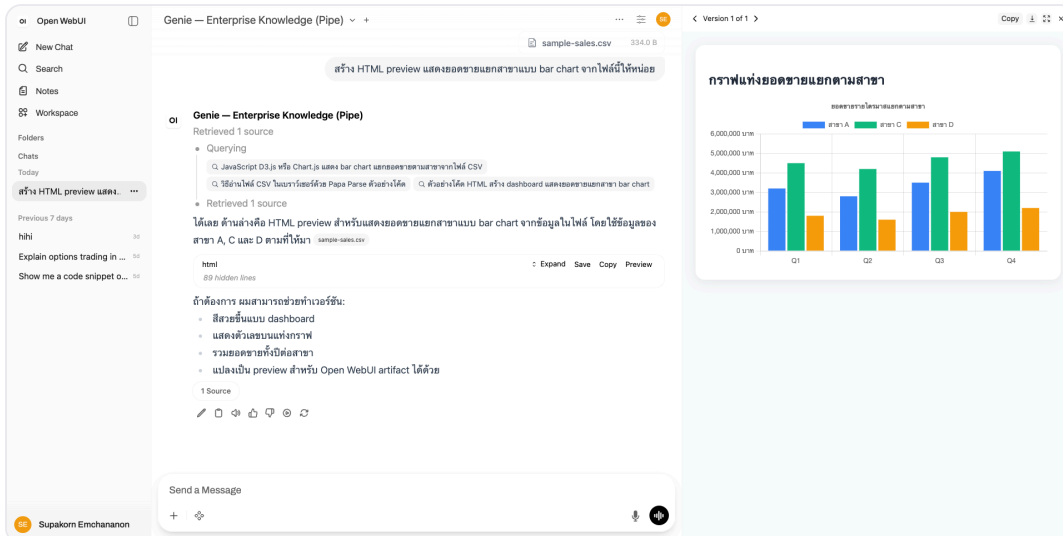
- ลากไฟล์ PDF, Excel, Word, CSV มาวางในแชท
- Genie อ่านเอกสารให้ แล้วถามรายละเอียดได้เลย
- ลอง: "สรุปเอกสารนี้ให้หน่อย" หรือ "หาข้อมูลเกี่ยวกับ X ในไฟล์นี้"

ตัวอย่าง: CSV → HTML Preview

อัปโหลดไฟล์ CSV ยอดขาย แล้วให้ Genie สร้าง HTML preview แบบ Bar Chart ให้ดูทันที — ไม่ต้องเขียนโค้ดเอง!



ผลลัพธ์ — Genie สร้าง Bar Chart ด้วย Chart.js ให้ดูในแชท:



ฟังก์ชันพื้นฐาน

เลือกโมเดล

- คลิกชื่อโมเดล → เลือกได้ตามต้องการ
- "Genie — Enterprise Knowledge" = AI องค์กร
- "MyAssistant" = GPT ทั้งหมด

ตั้งค่า

- ปุ่ม Controls — ปรับอุณหภูมิ, System Prompt
- Available Tools — ดู Tools ที่เปิดใช้
- Voice Input — พิมพ์ด้วยเสียง

ประวัติแชท

- ดูประวัติแชทที่ sidebar ด้านซ้าย
- กด Search เพื่อค้นหาแชทเก่า
- ตั้งชื่อแชทเพื่อจัดการง่ายขึ้น

แชร์แชท

- แชร์ประวัติแชทให้เพื่อนร่วมทีม
- Export แชทเป็นไฟล์
- Import แชทจากไฟล์

Tips & Tricks

ควรถาม

- "ตั้ง MFA ในมือถือยังไง"
- "สิทธิ์ลาที่วัน"
- "นโยบาย PDPA"
- "ต่อ VPN ไม่ได้ทำไง"
- "OT ได้เท่าไร"

ไม่ควรถาม

- ข้อมูลลับ/ความลับทางการค้า
- รหัสผ่าน หรือข้อมูลส่วนตัว
- ข้อมูลที่ไม่เกี่ยวกับองค์กร
- คำถามที่ไม่เป็นความจริง/เข้าใจผิด

Use Cases ตามแผนก (Enterprise AI)

ตัวอย่างการประยุกต์ใช้ Genie สำหรับธุรกิจหาทรัพยากร — จาก AI Use Cases โดย Gemini:

ฝ่ายขาย

- **Sales Brief** — สรุปข้อมูลร้านค้าก่อนลงพื้นที่
- **Promotion Analyzer** — วิเคราะห์โปรโมชั่นที่ใช้

การตลาด

- **Local Content** — คอนเทนต์ภาษาใต้
- **Southern Campaign** — กิจกรรมออนกราวด์

ฝ่ายผลิต

- **Demand Planner** — วางแผนผลิตตามดีมานด์
- **Maintenance Co-pilot** — ช่อมบำรุงเครื่องจักร

ฝ่ายขนส่ง

- **Route Optimizer** — เส้นทางประหยัดน้ำมัน
- **POD Agent** — ตรวจสอบใบส่งสินค้า

ฝ่าย HR

- **HR Benefits Bot** — สวัสดิการ 24 ชม.
- **Recruitment Agent** — กรองเรซูเม่

ฝ่าย IT

- **IT Helpdesk** — แก้ปัญหาไอทีเบื้องต้น
- **Cybersecurity Monitor** — ตรวจจับความผิดปกติ

ฝ่ายการเงิน

- **Expense Auditor** — ตรวจสอบใบเสร็จ
- **Credit Monitor** — ติดตามหนี้สินเชื่อ

ลูกค้าสัมพันธ์

- **B2B Ordering Bot** — สั่งซื้อผ่าน Line
- **Cooler Service Agent** — แจ้งซ่อมตู้แช่

 ใช้ Prompts (/command) สำหรับงานที่ต้องการคำสั่งเฉพาะ — Skills จะทำงานอัตโนมัติเมื่อเจอคำถามที่ตรง

วิธีใช้ Tools และ Prompts

- **Tools** — เปิด/ปิด Tool ได้ที่ Controls → Available Tools (เช่น Enterprise Search)
- **Prompts** — พิมพ์ / ในแชท → เลือก Prompt ที่เซฟไว้ → ใช้ซ้ำได้ทันที
- **Knowledge** — ถ้าต้องการให้ Genie ใช้ Knowledge Base ให้เปิดใน Controls
- **Code Interpreter** — เปิดให้ Genie เขียนและรันโค้ด Python วิเคราะห์ข้อมูล
- **Web Search** — เปิดให้ Genie ค้นหาข้อมูลจาก internet

Knowledge Base

จัดการฐานความรู้ (Knowledge Base) สำหรับให้ Agent ใช้อ้างอิง

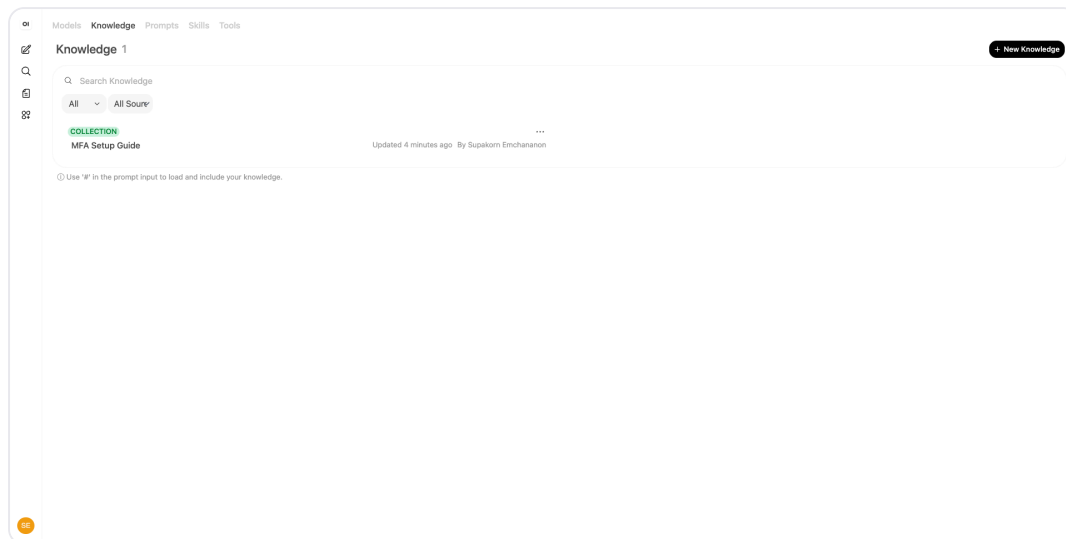
Knowledge Base คืออะไร?

Knowledge Base (KB) คือคลังเอกสารที่ Agent สามารถดึงมาอ่านเพื่อตอบคำถามได้ — เหมือนมีแฟ้มเอกสารไว้ให้ AI ค้นหา

- รองรับไฟล์: **PDF, DOCX, TXT, CSV, MD**
- สามารถอัปโหลดทีละไฟล์ หรือทั้งโฟลเดอร์
- เนื้อหาจะถูก chunk และ embed เป็น vector อัตโนมัติ
- เวลาถาม Agent จะค้นหาที่ KB ก่อน แล้วค่อยตอบ

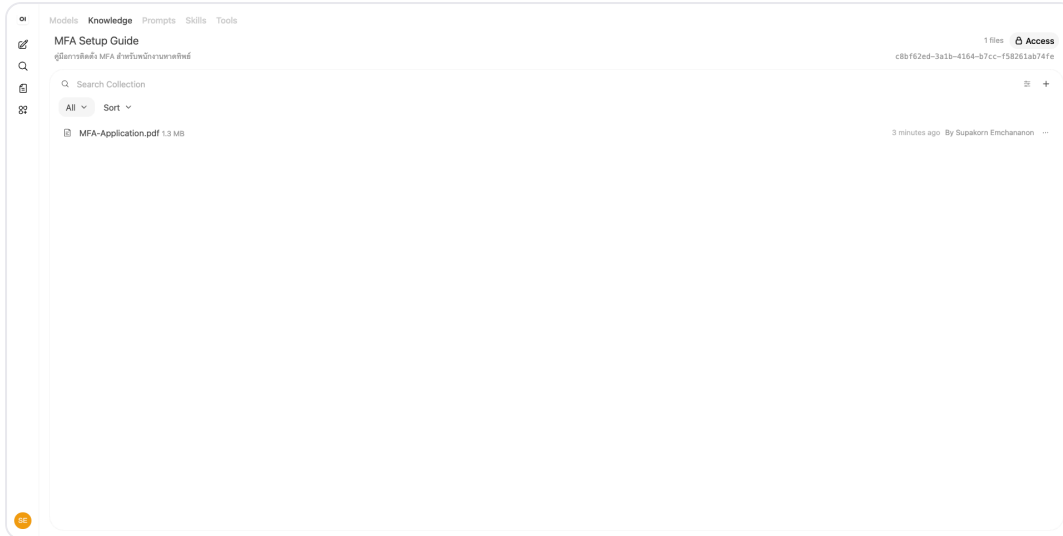
1 ไปที่ Workspace → Knowledge

คลิก **Workspace → Knowledge** — จะเห็นรายการ Knowledge Bases ที่สร้างไว้



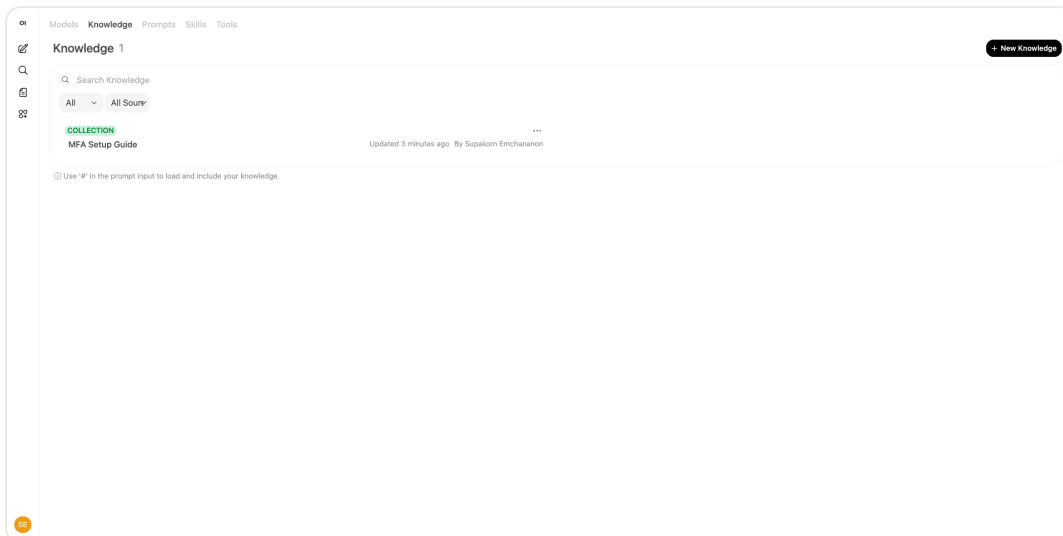
2 สร้าง Knowledge Base ใหม่

- กด **New Knowledge** → ตั้งชื่อ เช่น **MFA Setup Guide**
- เลือก **Public** เพื่อให้ทุกคนใช้งานได้
- กด **Create Knowledge** → จากนั้นอัปโหลดไฟล์ PDF/DOCX



3 ตัวอย่าง: MFA Setup Guide

Knowledge base "MFA Setup Guide" ที่มีไฟล์ PDF คู่มือการติดตั้ง MFA อัปโหลดไว้ — พร้อมให้ Agent ใช้อ้างอิง





แนะนำ



ใช้กับ Agent Enterprise

- ประกาศ HR, ระเบียบ, คำสั่ง
- คู่มือ IT, นโยบายความปลอดภัย
- เอกสาร Open WebUI



ข้อควรรู้

- ขนาดไฟล์ไม่ควรเกิน 50MB
- ชื่อไฟล์ควรสื่อความหมาย (อังกฤษ)
- KB ที่อัปเดตแล้วต้องรอ re-index



Prompts

จัดการ Prompt Templates สำหรับเรียกใช้ซ้ำได้



Prompts คืออะไร?

Prompts คือ เทมเพลตคำสั่ง ที่เซฟไว้ใช้กับ Agent ได้ทันที โดยไม่ต้องพิมพ์ซ้ำ — คล้ายๆ กับ shortcut คำสั่ง

ตัวอย่าง Prompt Template

ชื่อ: สรุปเอกสาร

Prompt: ช่วยสรุปเอกสารต่อไปนี้เป็นภาษาไทย

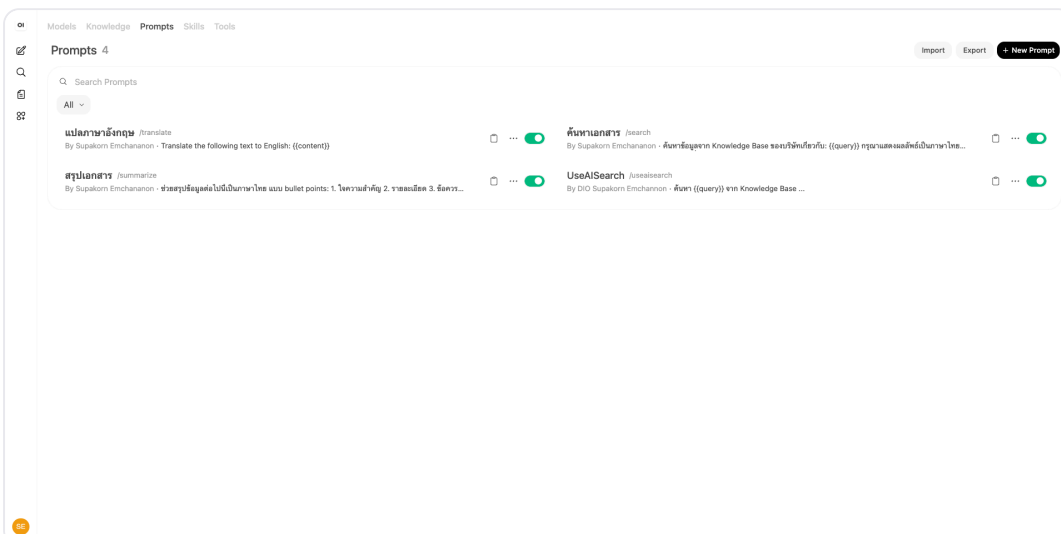
แบบ bullet points แยกเป็นหัวข้อ:

1. ใจความสำคัญ
2. รายละเอียด
3. ข้อควรปฏิบัติ

{content}

1 ไปที่ Workspace → Prompts

คลิก **Workspace** → **Prompts** — จะเห็นรายการ Prompt Templates ที่สร้างไว้



2 สร้าง Prompt

- **Prompt Name:** ตั้งชื่อที่จำง่าย (เช่น สรุปเอกสาร , ค้นหาเอกสาร)
- **Command:** พิมพ์ /summarize หรือ /search เพื่อใช้ในแชท
- **Prompt Content:** ข้อความคำสั่ง ใช้ {{variable}} เป็นตัวแปร

ตัวอย่าง Prompt ที่สร้างไว้ — ทั้งหมด 14 รายการ (Public)

ชื่อ	คำสั่ง	คำอธิบาย	แผนก
Sales Brief	/sales-brief	สรุปข้อมูลร้านค้าก่อนลงพื้นที่ขาย	🛒 ขาย
Promotion Analyzer	/promo-analyze	วิเคราะห์โปรโมชั่นเครื่องดื่มที่เหมาะสม	📢 การตลาด
Local Content Creator	/local-content	สร้างสื่อการตลาดภาษาไทยผสมภาษาใต้	📢 การตลาด
Production Demand Planner	/production-plan	วางแผนการผลิตตามดีมานด์และสภาพอากาศ	🏭 ผลิต
Machine Maintenance Guide	/maintenance-guide	ค้นหาวิธีซ่อมบำรุงเครื่องจักร	🏭 ผลิต
Route Optimizer	/route-optimize	วางแผนทางจัดส่งประหยัดน้ำมัน	🚚 ขนส่ง
HR Benefits Assistant	/hr-benefits	ตอบคำถามสวัสดิการพนักงาน 24 ชม.	👥 HR
IT Helpdesk	/it-helpdesk	แก้ไขปัญหาไอทีเบื้องต้น	💻 IT
Expense Auditor	/expense-check	ตรวจสอบใบเสร็จค่าใช้จ่ายตามนโยบาย	💰 การเงิน
Credit & Collection	/credit-collect	วิเคราะห์วงเงินสินเชื่อและติดตามหนี้	💰 การเงิน

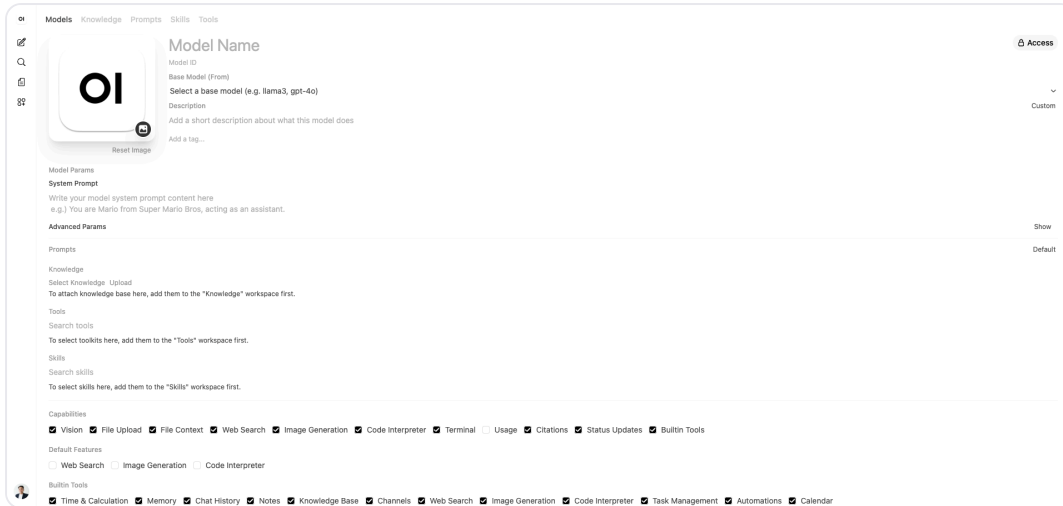
💡 พิมพ์ / ในแชทแล้วเลือก Prompt ที่ต้องการใช้ — รองรับตัวแปร {{variable}}

🤖 ขั้นตอนสร้าง Agent

สร้าง Model ที่มี Tool ติดตัว — พร้อมใช้งาน

1 Workspace → Models → เลือก Model

คลิก **Workspace → Models** → เลือก model ที่ต้องการ (เช่น `deploy-gpt-5.4-mini`) หรือสร้างใหม่



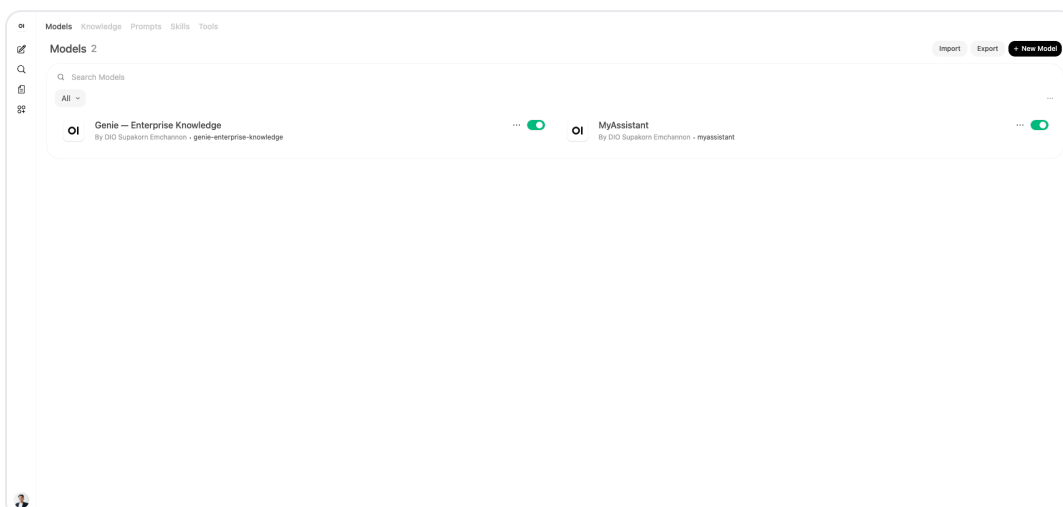
2 เปิด Tools

เลื่อนลงมาที่ส่วน **Tools** → ค้นหา → เลือก **"Enterprise Search"**

จากนั้นเลือก **Native Mode (Agentic Mode)** — model จะเรียก Tool อัตโนมัติเวลาต้องการค้นหาเอกสาร

3 Save

กด **Save** → กลับมาดู Models list จะเห็น Model พร้อมใช้งาน



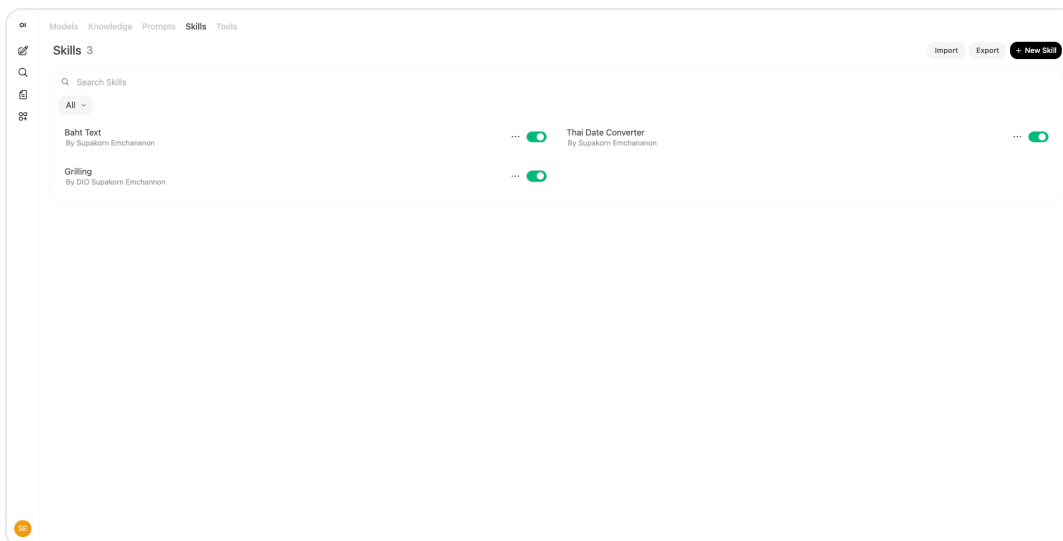
Skills คืออะไร?

Skills คือ ความสามารถเฉพาะทาง ที่เพิ่มให้ Agent ได้ เช่น การคำนวณขั้นสูง, การดึงข้อมูลจาก API ภายนอก, การทำงานกับระบบเฉพาะ

- ต่างจาก Tools — Skills จะถูกเรียกใช้ อัตโนมัติ โดย Agent (ไม่ต้องเลือกเอง)
- เป็นเหมือน "พรสวรรค์" ที่ Agent มีติดตัว
- สร้างด้วยภาษา Python เช่นเดียวกับ Tools

1 ไปที่ Workspace → Skills

คลิก **Workspace** → **Skills** — จะเห็น Skills ที่สร้างไว้



2 สร้าง Skill

- **Skill Name:** ชื่อ Skill (เช่น `Thai Date Converter`)
- **Skill ID:** `thai-date-converter`
- **Description:** อธิบายว่า Skill นี้ทำอะไร
- **Instructions:** เขียนคำแนะนำในรูปแบบ Markdown — บอกวิธีการทำงานให้ Agent ทำตาม

 ตัวอย่าง Skills ที่สร้างไว้ — ทั้งหมด 13 รายการ (Public)

Skill	คำอธิบาย	แผนก
Sales Pre-visit Agent	สรุปข้อมูลร้านค้าก่อนลงพื้นที่ + Next-Best-Action	 ขาย
Promotion Optimizer	วิเคราะห์โปรโมชั่นและแนะนำรูปแบบที่เหมาะสม	 การตลาด
Southern Content Creator	สร้างสื่อการตลาดภาษาใต้สำหรับเครื่องดื่ม	 การตลาด
Production Demand Planner	คาดการณ์ปริมาณการผลิตตามดีมานด์	 ผลิต
Maintenance Co-pilot	ค้นหาคู่มือซ่อมบำรุงเครื่องจักร	 ผลิต
Route Optimizer	วางแผนทางจัดส่งอัจฉริยะ	 ขนส่ง
HR Benefits Bot	ตอบคำถามสวัสดิการและสิทธิพนักงาน	 HR
IT Helpdesk Assistant	ช่วยแก้ไขปัญหาไอทีเบื้องต้น	 IT
Expense Auditor	ตรวจสอบใบเสร็จค่าใช้จ่ายตามนโยบาย	 การเงิน
Credit Monitor	วิเคราะห์หนี้เงินสินเชื่อและติดตามหนี้	 การเงิน

Skills จะถูกเรียกใช้โดย Agent อัตโนมัติเมื่อเจอคำถามที่ตรงกับความสามารถ — ไม่ต้องพิมพ์คำสั่งใด ๆ

3 เพิ่ม Skill ให้ Agent

เวลา สร้าง/แก้ไข Model (Workspace → Models):

- เลื่อนลงมาที่ส่วน Skills
- ค้นหาและเลือก Skill ที่ต้องการ
- Agent จะใช้ Skill โดยอัตโนมัติเมื่อเจอคำถามที่ตรง

 เปรียบเทียบ Tools vs Skills

คุณสมบัติ	 Tools	 Skills
การเรียกใช้	ต้องเลือกตอนถาม หรือให้ Agent เลือก	เรียกอัตโนมัติ
Class name	<code>class Tools:</code>	<code>class Skills:</code>
เหมาะกับ	ค้นหา, คำนวณ, API calls	แปลงข้อมูล, จัดรูปแบบ, validate
เปิด/ปิด	เลือกเปิดเฉพาะ Model	เพิ่มใน Model แล้วทำงานอัตโนมัติ

💡 Use Case ตามแผนก

Skills ข้างต้นครอบคลุมทุกแผนกของหาดทิพย์ — จาก Gemini AI Use Cases:

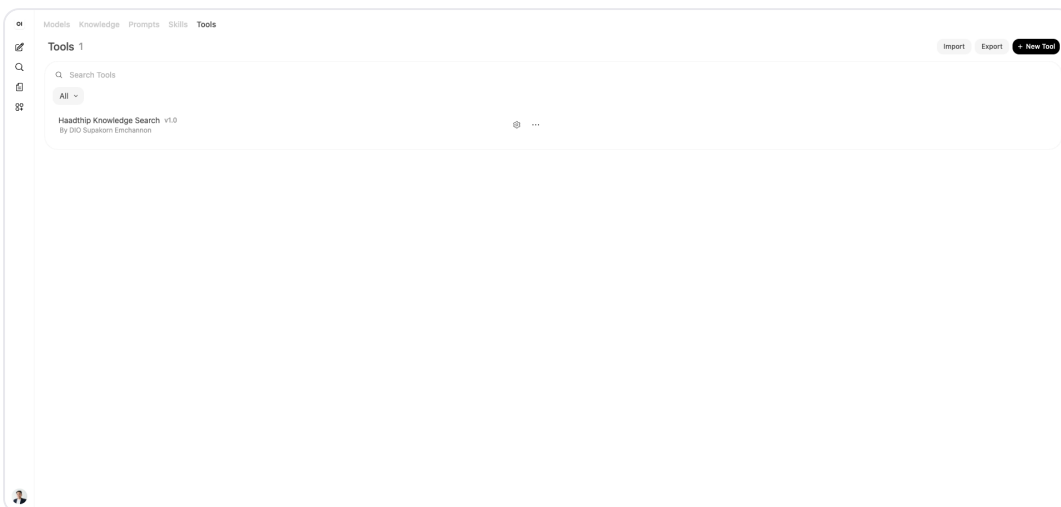
- 🛒 ฝ่ายขาย: รู้จักร้านค้าก่อนไปพบ, วิเคราะห์โปรโมชั่น
- 🏭 ฝ่ายผลิต: วางแผนการผลิต, ซ่อมบำรุงเครื่องจักร
- 🚚 ฝ่ายขนส่ง: วางเส้นทาง, เพิ่มประสิทธิภาพน้ำมัน
- 👤 ฝ่าย HR: สวัสดิการ, สิทธิพนักงาน ตลอด 24 ชม.
- 💻 ฝ่าย IT: Helpdesk อัปเดต, Cybersecurity
- 💰 ฝ่ายการเงิน: ตรวจสอบค่าใช้จ่าย, ติดตามหนี้
- 📢 ฝ่ายการตลาด: คอนเทนต์ภาษาใต้, โปรโมชั่นเฉพาะพื้นที่

🔧 ขั้นตอนเพิ่ม Tool

เพิ่ม Enterprise Search Tool สำหรับค้นหาเอกสารภายใน

1 ไปที่ Workspace → Tools

คลิกเมนู **Workspace** → แท็บ **Tools** → กด **New Tool**



2 กรอกข้อมูล

- **Tool Name:** Enterprise Search
- **Tool ID:** enterprise_search
- **Description:** Search internal docs, HR policies, and Open WebUI help via Azure AI Search.

ModelsKnowledgePromptsSkillsTools

< Tool Name

Access

Tool ID

Tool Description

```
1 import os
2 import requests
3 from datetime import datetime
4 from pydantic import BaseModel, Field
5
6 class Tools:
7     def __init__(self):
8         pass
9
10    # Add your custom tools using pure Python code here, make sure to add type hints and descriptions
11
12    def get_user_name_email_and_id(self, __user__: dict = {}) -> str:
13        """
14        Get the user name, Email and ID from the user object.
15        """
16
17        # Do not include a description for __user__ as it should not be shown in the tool's specification
18        # The session user object will be passed as a parameter when the function is called
19
20        print(__user__)
21        result = ""
22
23        if "name" in __user__:
24            result += f"User: {__user__['name']}"
25        if "id" in __user__:
26            result += f" (ID: {__user__['id']})"
27        if "email" in __user__:
28            result += f" (Email: {__user__['email']})"
29
30        if result == "":
31            result = "User: Unknown"
32
33        return result
34
35    def get_current_time(self) -> str:
36        """
37        Get the current time in a more human-readable format.
38        """
39
```

Tools are a function calling system with arbitrary code execution
— don't install random tools from sources you don't trust.

Save

3 วางโค้ด Tool

เปิดไฟล์ Tool → copy ทั้งหมด → วางใน Code Editor

```
"""
title: Enterprise Search (Tool)
author: Haadthip DIO
version: 1.0
required_open_webui_version: 0.1.0
"""

import json, os
from typing import Any
import httpx
from pydantic import BaseModel, Field

class Tools:
    class Valves(BaseModel):
        search_endpoint: str = Field(
            default="https://srch-entchat-poc-sand.search.windows.net")
        search_api_key: str = Field(
            default=os.environ.get("AZURE_SEARCH_ADMIN_KEY", ""))
        index_name: str = Field(default="enterprise-docs-idx")
        max_results: int = Field(default=5, ge=1, le=10)
        search_timeout: int = Field(default=30)

    def __init__(self):
        self.valves = self.Valves()

    async def search_internal_docs(self, query: str) -> str:
        """Search IT policies, MFA, VPN, Email, Meeting, HR"""
        return await self._search(query, "corporate,hr-policies")

    async def search_hr_policies(self, query: str) -> str:
        """Search HR: leave, benefits, training, PDPA, etc."""
        return await self._search(query, "hr-policies")

    async def search_openwebui_help(self, query: str) -> str:
        """Search Open WebUI docs: API, features, setup"""
        return await self._search(query, "openwebui-docs")

    async def _search(self, query: str, corpus_filter: str) -> str:
        key = self.valves.search_api_key or os.environ.get("AZURE_SEARCH_ADMIN_KEY", "")
        if not key: return "⚠️ Missing search_api_key"
        url = f"{self.valves.search_endpoint}/indexes/{self.valves.index_name}/docs/search?api-v
        corpuses = [c.strip() for c in corpus_filter.split(",")]
        filter_str = f"corpus eq '{corpuses[0]}'" if len(corpuses) == 1 \
            else "(" + " or ".join(f"corpus eq '{c}'" for c in corpuses) + ")"
        body = {
            "search": query, "filter": filter_str, "queryType": "semantic",
            "semanticConfiguration": "semantic-config",
            "searchFields": "content,summary,file_name,category",
            "select": "file_name,corpus,category,content,summary,meta",
            "top": self.valves.max_results, "captions": "extractive",
            "answers": "extractive|count-1",
        }
        async with httpx.AsyncClient(timeout=self.valves.search_timeout) as client:
            try:
                resp = await client.post(url, json=body,
                    headers={"Content-Type": "application/json", "api-key": key})
                resp.raise_for_status(); data = resp.json()
            except Exception as e: return f"⚠️ Search error: {e}"
        answer_text = ""
        if data.get("@search.answers"):
```

```

answer_text = data["@search.answers"][0].get("text", "")
docs = []
for doc in data.get("value", []):
    fname, cat = doc.get("file_name", "Untitled"), doc.get("category", "")
    summary, content = doc.get("summary", ""), doc.get("content", "")[:400]
    caption = ""
    if doc.get("@search.captions"):
        caption = doc["@search.captions"][0].get("text", "")
    text = caption or content
    header = f"[{cat}] {fname}"
    if summary and summary not in text: text = f"{summary}\n{text}"
    docs.append(f"{header}\n{text}")

result = "\n\n---\n\n".join(docs) if docs else "ไม่พบเอกสาร"
if answer_text: result = f"📄 **Answer:** {answer_text}\n\n---\n\n{result}"
return result if result.strip() else "ไม่พบเอกสารที่เกี่ยวข้อง"

```

Enterprise Search (Tool)

```

# Extract answer if available
answer_text = ""
if data.get("@search.answers"):
    answer_text = data["@search.answers"][0].get("text", "")

# Format results
docs = []
for doc in data.get("value", []):
    file_name = doc.get("file_name", "Untitled")
    category = doc.get("category", "")
    summary = doc.get("summary", "")
    content = doc.get("content", "")[:400]
    caption = ""
    if doc.get("@search.captions"):
        caption = doc["@search.captions"][0].get("text", "")
    text = caption or content
    header = f"[{category}] {file_name}"
    if summary and summary not in text:
        text = f"{summary}\n{text}"
    docs.append(f"{header}\n{text}")

result = ""

".join(docs) if docs else "ไม่พบเอกสาร"

# Prepend answer if available
if answer_text:
    result = f"📄 **Answer:** {answer_text}\n\n---\n\n{result}"

return result if result.strip() else "ไม่พบเอกสารที่เกี่ยวข้อง"

```

Functions allow arbitrary code execution.
— don't install random functions from sources you don't trust.

Save

4 Save + Confirm

กด **Save** → เช็ค *"I acknowledge..."* → กด **Confirm**

Enterprise Search (Tool)

```

# Extract answer if available
answer_text = ""
if data.get("@search.answers"):
    answer_text = data["@search.answers"][0].get("text", "")

# Format results
docs = []
for doc in data.get("value", []):
    file_name = doc.get("file_name", "Untitled")
    category = doc.get("category", "")
    summary = doc.get("summary", "")
    content = doc.get("content", "")[:400]
    caption = ""
    if doc.get("@search.captions"):
        caption = doc["@search.captions"][0].get("text", "")
    text = caption or content
    header = f"[{category}] {file_name}"
    if summary and summary not in text:
        text = f"{summary}\n{text}"
    docs.append(f"{header}\n{text}")

result = ""

".join(docs) if docs else "ไม่พบเอกสาร"

# Prepend answer if available
if answer_text:
    result = f"📄 **Answer:** {answer_text}\n\n---\n\n{result}"

return result if result.strip() else "ไม่พบเอกสารที่เกี่ยวข้อง"

```

Functions allow arbitrary code execution.
— don't install random functions from sources you don't trust.

Enterprise Search (Tool)

Confirm your action

Check the following settings:

- Confirm the function name and description.
- Confirm the function code.

By clicking 'Confirm', you acknowledge that you have read and understood the function code and its potential impact on the system. You also acknowledge that you have read and understood the function code and its potential impact on the system.

Cancel Confirm

5 ตรวจสอบ

กลับมา Tools list จะเห็น **Enterprise Search** (Tools: 2)

